

Physics Informed Neural Networks with Learnable Scaling for Feature Engineering

Anonymous Authors¹

Abstract

Spectral methods are widely used in machine learning for feature engineering and representation learning. Typically, such approaches rely on defining a kernel or an integral/differential operator whose eigenfunctions are then used to construct data-driven features. In practice, however, the performance of spectral methods is highly sensitive to the choice of kernel, feature scaling, and associated hyperparameters, often requiring substantial manual tuning.

In this work, we propose an alternative spectral framework that alleviates these limitations. Our approach is based on eigenfunctions of a modified Dirichlet energy functional, defined as the expected squared norm of the gradient of a function. The key modification consists in introducing a learnable linear transformation of the gradient within the Dirichlet energy, allowing the spectral structure to be adapted directly from data.

This formulation provides several important benefits. First, it relaxes the classical manifold hypothesis by interpolating between differential-geometric operators, such as the Laplace–Beltrami operator, and purely statistical objects defined by energy spectra. Second, it significantly reduces the need for manual hyperparameter tuning and feature scaling, as the relevant transformations are learned automatically. Finally, the proposed method yields compact and expressive data representations that are naturally aligned with downstream tasks, such as classification.

We empirically validate the proposed approach on six benchmark datasets spanning binary and multi-class classification: Two Moons, Circles,

HTRU2, MNIST, Fashion-MNIST, and CIFAR-10. The EFDO method with diagonal metric achieves 94.53% validation accuracy on MNIST and 85.50% on CIFAR-10 features, surpassing the NeuralEF baseline (Deng et al., 2022) by +12.0 pp and +9.4 pp respectively (NeuralEF: 82.52% / 76.05%, $K=16$ eigenfunctions). Training curves and learned eigenfunction visualisations confirm stable convergence and meaningful geometric structure.

1. Introduction

Contributions. The main contributions of this work are as follows:

- We introduce a learnable modification of the Dirichlet energy that generalizes classical spectral operators.
- We show that the resulting eigenfunctions provide compact and task-adaptive representations.
- We demonstrate empirically that the proposed approach reduces sensitivity to feature scaling and hyperparameter tuning.
- We evaluate the approach on six benchmarks (Two Moons, Circles, HTRU2, MNIST, Fashion-MNIST, CIFAR-10) and compare against Random Forest, Logistic Regression, and NeuralEF baselines.
- We introduce and compare three metric parametrisations—identity (OFF), diagonal (DIAG), and Trotter-product rotation (TROTTER)—analysing their effect on representation quality.

Artificial Neural Networks (ANN) Rosenblatt (1958) are routinely utilized to solve real-world problems: medical image segmentation Ajmal et al. (2018), medical diagnostics Amato et al. (2013), photo and video processing and enhancement Ahmadi & Patras (2016), face recognition Le (2011), text processing Chang et al. (2023) etc. It has been shown in recent decades that ANN can handle such problems as numerical solution for PDEs and for calculation of differential and integral operators eigenfunctions Jin

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

et al. (2020). In the current work we propose a novel approach for calculation of the Dirichlet Energy [Evans \(2022\)](#) eigenfunctions.

The main motivation for this work is provided by Graph-Laplacian methods [Gomez-Chova et al. \(2008\)](#). In this family of methods the graph is constructed based on the available data and the spectrum of Laplacian associated with this graph is computed [Kunegis et al.](#) Eigen-functions of that Laplace operator have extremely useful properties especially when data is sampled from a certain manifold. First of all, it can be shown that the number of components of the graph is greater or equal to the dimension of Laplace operator kernel [Belkin & Niyogi \(2008\)](#). This fact gives rise to a variety of clustering algorithms including the well-known spectral clustering method [Ng et al. \(2001\)](#). In addition to that, eigenfunctions of the Laplacian allows one to perform Fourier analysis in the space of functions on the graph vertices [Kostopoulos et al. \(2018\)](#) The latter gives rise to the variety of algorithms for semi-supervised learning [Sheikhpour et al. \(2018\)](#). In other words, eigenfunctions of Laplace operator on Graph can be successfully utilized to propagate labels from few labeled samples to the entire dataset [Calder et al. \(2020\)](#).

The reason for such remarkable performance is that eigenfunctions of Laplace operator on the graph converge to the eigenfunctions of Laplace-Beltrami operator if data is sampled from the manifold as it has been shown in the pioneering work [Belkin & Niyogi \(2008\)](#). Therefore, the information about intrinsic geometry of the data is passed through eigenfunctions of Graph-Laplacian. That is one of the reasons for the outstanding performance of Graph-Laplacian based methods in various machine learning tasks.

Despite obvious advantages of Graph-Laplacian methods, there are at least two factors that limit the application of the technique concerned. First of all, the cost of the linear operator eigenfunctions calculation increases rapidly with the growth of the dataset [Ford \(2015\)](#). The second issue with Graph-Laplacian based methods is inference. In order to compute model predictions at the point that does not belong to the training dataset graph and its eigenfunctions should be recomputed [Belkin & Niyogi \(2008\)](#). In other words, computational costs of model training and inference are comparable, which is not affordable for real-time applications.

The success of the Graph-Laplacian based method demonstrates that knowledge of differential operator eigenfunctions is beneficial for machine learning tasks. However, issues associated with high computational cost for big datasets sets severe limitations on the Graph-Laplacian based methods applications. In this work we are proposing the novel approach for approximation of the Dirichlet Energy eigenfunctions via ANN. In the proposed method eigenfunc-

tions are computed sequentially based on the training data. Once computed, ANN approximation for eigenfunctions can be directly utilized without need for extra training. Such approach combines advantages of Graph-Laplacian based methods and has almost zero cost for inference in comparison with training.

Solutions for eigen problems are of great practical value. For instance, eigen-vectors and/or eigenfunctions are routinely utilized in the modern machine learning in such algorithms like Principal Components Analysis (PCA) [Halko et al. \(2011\)](#), Spectral-Clustering [Ng et al. \(2001\)](#), Cocktail Party Problem [Sejnowski \(2018\)](#) etc. In addition to that, Eigen-functions of differential operators are extremely useful for analysis of physical systems driven by elliptical or parabolic Partial Differential Equations (PDE) [Ben-Shaul et al. \(2023\)](#) as well as in Quantum Mechanics [Choo et al. \(2020\)](#).

The canonical approach for eigenfunctions approximation of Laplace-like operators is to introduce spatial discretization to compute the eigen-function approximately [M. A. ERICKSON & LAUB \(1995\)](#). Application of ANNs to eigen-vector problems is a growing area of research, where both practical [Choo et al. \(2020\)](#); [Lagaris et al. \(1997\)](#) and fundamental questions such as learnability [Simon et al. \(2022\)](#); [Ben-Shaul et al. \(2023\)](#) are considered.

Automatic differentiation in modern deep learning frameworks provide tools for mesh-free approximation of eigenfunctions. The latter is possible due to automatic differentiation, which is an essential part of modern ANNs. Therefore, moderate programming effort is required to compute various order partial derivatives of quite generic ANN with respect to the input features. Given the fact that eigen-function calculation can be reduced to the minimization of Rayleigh ratio [Feld et al. \(2019\)](#), the ANN approximation problem can be formulated as the optimization problem under orthogonality and normalization constraints [Deng et al. \(2022\)](#).

In the present paper we propose the novel approach for eigenfunctions approximation with ANNs. We consider the Dirichlet Energy, which is a simple version of Laplace-Beltrami operator that has similar properties. In the novel approach eigenfunctions are computed sequentially and calculation of each eigen-function is formulated as an optimization problem under orthogonality and normalization constraints. The presented approach is generic, so that any given eigen function can be approximated in principle. In addition to that, the problem of overfitting is considered. It is demonstrated via numerical experiments that for large enough ANNs eigen-values of approximated eigenfunctions can be lower than the theoretical bound. The overfitting effect for eigenfunctions is demonstrated on the synthetic dataset, where the exact solution is known. Data augmentation techniques to address the issue concerned are proposed.

The rest of the paper is organized as follows: in the section 2 details of the novel approach are explained. In the section 3 numerical examples with different datasets are provided followed by summary and concluding remarks.

2. Methodology.

In the present section we describe the key ideas of the approach introduced. First of all, we start with the loss-function derivation as a weighted sum of three different terms. Secondly, we describe dynamic procedure for loss-functions weights calculation. Thirdly, we explain how linear transformation of function gradients is implemented. Finally, we provide the details about pretraining procedure that is utilised in the current work to improve the convergence of the algorithm.

2.1. Loss-Function.

In the present work we follow classical way for calculation of eigenfunctions through minimisation of Rayleigh quotient subjected to orthogonality conditions. In other words, we suppose that there is a probability distribution with the density $p(\mathbf{x})$. This probability distribution is utilised to introduce the following notion of the inner product:

$$\langle f_1, f_2 \rangle_0 = \int f_1(\mathbf{x}) f_2(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} \quad (1)$$

Here f_1 and f_2 are functions with finite second moments.

Alternative product or bilinear form can be naturally introduced:

$$\langle f_1, f_2 \rangle_{de} = \int \sum_{a=1}^d \frac{\partial f_1(\mathbf{x})}{\partial x_a} \frac{\partial f_2(\mathbf{x})}{\partial x_a} p(\mathbf{x}) d\mathbf{x} \quad (2)$$

Here d is the dimension of $\mathbf{x}$.

This product becomes a regular Dirichlet Energy for $f_1 = f_2$:

$$\langle f, f \rangle_{de} = \int \sum_{a=1}^d \frac{\partial f(\mathbf{x})}{\partial x_a} \frac{\partial f(\mathbf{x})}{\partial x_a} p(\mathbf{x}) d\mathbf{x} = \int \|\nabla f(\mathbf{x})\|^2 p(\mathbf{x}) d\mathbf{x} \quad (3)$$

In the present work we introduce a learnable matrix $\mathbf{A}(\mathbf{x})$ to modify the Dirichlet energy and the corresponding bilinear form:

$$\langle f_1, f_2 \rangle_{mde} = \int \sum_{a_1, a_2, b_1, b_2=1}^d A_{a_1 b_1} \frac{\partial f_1(\mathbf{x})}{\partial x_{b_1}} A_{a_2 b_2} \frac{\partial f_2(\mathbf{x})}{\partial x_{b_2}} p(\mathbf{x}) d\mathbf{x} \quad (4)$$

The Modified Dirichlet Energy (MDE) can be computed as

following:

$$\langle f, f \rangle_{mde} = \int \|\mathbf{A}(\mathbf{x}) \nabla f(\mathbf{x})\|^2 p(\mathbf{x}) d\mathbf{x} \quad (5)$$

The idea is to compute first n eigenfunctions $\phi_1(\mathbf{x}), \dots, \phi_n(\mathbf{x})$ via Rayleigh ratio minimisation under orthonormality constraint. Matrix $\mathbf{A}(\mathbf{x})$ is learnt to adjust features in such a way that the accuracy of logistics regression solved with that features is maximized.

In other words, we have a loss-function that corresponds to the MDE:

$$\mathcal{L}_{mde} = \sum_{\alpha=1}^n \int \|\mathbf{A}(\mathbf{x}) \nabla f(\mathbf{x})\|^2 p(\mathbf{x}) d\mathbf{x} \quad (6)$$

There is a loss-function that enforces orthonormality constraint:

$$\mathcal{L}_{orth} = \sum_{\alpha, \beta=1}^n \left(\int (\phi_\alpha(\mathbf{x}) \phi_\beta(\mathbf{x}) - \delta_{\alpha\beta}) p(\mathbf{x}) d\mathbf{x} \right)^2 \quad (7)$$

Here $\delta_{\alpha\beta}$ is the Kronecker symbol or simply element of the identity matrix.

Features $\phi_1(\mathbf{x}), \dots, \phi_n(\mathbf{x})$ are utilised to do train the classifier:

$$\log(p(l|\mathbf{x})) \propto B_l + \sum_{\gamma=1}^n W_{l\gamma} \phi_\gamma(\mathbf{x}) \quad (8)$$

Here B is the bias, W is the matrix of weights and $\log(p(l|\mathbf{x}))$ is the probability of sample $\mathbf{x}$ to have a label (belong to class) l . Weights and bias are learnt from cross-entropy loss-function $\mathcal{L}_{class}$ minimisation.

Finally, all three loss-functions are combined together via weighting:

$$\mathcal{L} = \text{sg}(w_{orth}) \mathcal{L}_{orth} + \text{sg}(w_{class}) \mathcal{L}_{class} + \text{sg}(w_{mde}) \mathcal{L}_{mde} \quad (9)$$

Here sg is a stop-gradient operation that means that derivatives of weights should not be computed in the back-propagation step.

It is worth noting that integrals over the probability distribution is utilised in the loss-functions calculation as it follows from Eq. (6) and Eq. (7). In practice, the density of probability distribution is not known. Therefore, Monte-Carlo estimate is utilised to compute integrals in Eq. (6) and Eq. (7). In practice that means the replacement of the integral by averaging over the batch or entire dataset.

2.2. Loss-Function Weighting

In the present work we utilise the hierarchy of loss-functions:

- orthonormality
- cross-entropy for classification
- MDE loss-function

The objective is to learn representative features first and train the classifier with them. The final stage of MDE minimisation provides regularisation and improves the expressive power of constructed features.

The practical way to achieve the hierarchy of concern is dynamic weighting of loss-functions that is updated each iteration of loss-function minimisation:

$$\begin{cases} w_{\text{orth}} \propto 1, \\ w_{\text{class}} \propto \exp\left(-\mathcal{L}_{\text{orth}}/T_{\text{orth}}\right), \\ w_{\text{mde}} \propto \exp\left(-\max\left(\mathcal{L}_{\text{orth}}/T_{\text{orth}}, \mathcal{L}_{\text{class}}/T_{\text{class}}\right)\right). \end{cases} \quad (10)$$

Here T_{orth} and T_{class} are the target values of corresponding loss-functions. Specific values of T_{orth} and T_{class} are determined at the pretraining stage.

Eq. (10) utilises the exponential dumping of weights in the case of loss-functions high values. We do not know any fundamental reason for exponential functions. It simply appeared to be more robust in comparison with the functions of the type: $1/(1 + \mathcal{L}_{\text{orth}})$

It is clear from the Eq. (10) that the desired prioritisation of loss-function is achieved with such a weighting scheme: for instance, classifier is not trained at all if the value of $\mathcal{L}_{\text{orth}}$ exceeds a certain value. Similar holds for $\mathcal{L}_{\text{mde}}$.

2.3. Matrix Parametrisation

In the present work we represent matrix $\mathbf{A}(\mathbf{x})$ as a composition of rotation and scaling:

$$\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x})\mathbf{U}(\mathbf{x}) \quad (11)$$

Here $\mathbf{\Lambda}$ is the diagonal matrix and $\mathbf{U}(\mathbf{x})$ is rotation matrix.

Elements of $\mathbf{\Lambda}$ are trained as a fully connected neural network that is subjected to constraint:

$$\sum_{\alpha=1}^n \log(\lambda(\mathbf{x})) = 0 \quad (12)$$

We train Physics Informed Neural Network (PINNs) to compute $\mathbf{U}(\mathbf{x})$. More precisely, we train PINNs to solve for ODE for arbitrary initial condition and arbitrary skew-symmetric ω :

$$\frac{d\mathbf{v}}{dt} = \omega\mathbf{v} \quad (13)$$

For scalability reasons sparse parametrisation for ω is utilised. Basically, we assume that only two diagonals (one above and one below the principal diagonal) can have non-zero values.

Therefore, we pretrain PINN to compute $\exp(\omega)\mathbf{v}$ and learn ω as a function of $\mathbf{x}$

Implemented metric variants. In practice we study three variants of the matrix $\mathbf{A}(\mathbf{x})$:

- **Identity (OFF):** $\mathbf{A}(\mathbf{x}) = \mathbf{I}$, i.e. no transformation. This recovers the standard Dirichlet energy and serves as a baseline.
- **Diagonal (DIAG):** $\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x})$, where $\mathbf{\Lambda}$ is a diagonal matrix whose log-eigenvalues are zero-mean so that $\det \mathbf{\Lambda} = 1$:

$$\mathbf{\Lambda}(\mathbf{x}) = \text{diag}(\exp(z(\mathbf{x}) - \bar{z}(\mathbf{x}))), \quad z(\mathbf{x}) = \text{MLP}(\mathbf{x}) \in \mathbb{R}^d. \quad (14)$$

This corresponds to a learned axis-aligned rescaling of the gradient.

- **Trotter rotation (TROTTER):** the full factorisation $\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x})\mathbf{U}(\mathbf{x})$, where $\mathbf{U}$ is computed as a *Trotter product* of $d-1$ Givens rotations applied to consecutive index pairs:

$$\mathbf{U}(\omega) = R_{d-2}(\omega_{d-2}) \cdots R_1(\omega_1) R_0(\omega_0), \quad (15)$$

with $R_i(\omega_i)$ the rotation in the $(i, i+1)$ plane by angle $\omega_i = \pi \tanh(\text{MLP}(\mathbf{x})_i)$. The bounded-angle parametrisation ensures $\omega_i \in (-\pi, \pi)$ and that $\mathbf{U}$ is always an exact orthogonal matrix. The rotation is applied *first*: $\mathbf{A}\mathbf{v} = \mathbf{\Lambda}(\mathbf{U}\mathbf{v})$, so that $\|\mathbf{A}\mathbf{v}\|^2 = \sum_i \lambda_i^2 (\mathbf{U}\mathbf{v})_i^2$ depends on both $\mathbf{\Lambda}$ and $\mathbf{U}$, ensuring the rotation receives a non-zero gradient during training. To exploit GPU parallelism, the $d-1$ rotations are grouped into an even sweep (pairs $(0, 1), (2, 3), \dots$) followed by an odd sweep (pairs $(1, 2), (3, 4), \dots$), reducing the number of sequential Python operations from $O(d)$ to $O(1)$.

2.4. Pretraining.

To pretrain the neural network, we subsample a subset from training data and apply Graph-Laplacian approach to compute eigenfunctions on graph and to train the logistics regression with those features.

Therefore, we train our neural network to approximate the values of the Graph-Laplacian eigenfunctions on sampled data. Moreover, we utilise the value of cross-entropy loss of the logistics regression classifier as a target value T_{class} in the (10)

Note: In all experiments reported in Section 3, Graph-Laplacian pre-training is **disabled** and $T_{\text{class}} = 0.5$ is fixed. Pre-training remains a component of the algorithm and is available for future study.

3. Experimental Results

In the present section we consider four numerical examples. The first data set that we consider are points sampled uniformly from the unit circle. In this scenario eigenfunctions and corresponding eigenvalues are well-known. Ablation study is performed to demonstrate the significance of each step in the augmentation procedure. Therefore, this numerical example serves for the validation purposes and for the illustration of augmentation techniques.

In the next three examples we test our approach on three datasets: HTRU2 (Lyon et al., 2016; Lyon, 2016), MNIST (LeCun & Cortes, 2010) and Spotify songs dataset (Samoshyn, 2010). First several eigenfunctions and eigenvalues are computed for each dataset. Then the eigenfunctions concerned are utilized as input features for classification problems. Therefore, we evaluate the applicability of computed eigenfunctions to machine-learning problems.

In the examples with tabular datasets we consider computed eigenfunctions as features and study the applicability of these features to classification problems. The accuracy of classifiers trained on eigen-function values is compared with the accuracy of classifiers trained on the original features to ensure that eigenfunctions provide meaningful features. Two classifiers were considered: Random Forest (RF) and Logistic Regression (LR) implemented in the Scikit-Learn library (Pedregosa et al., 2011).

3.1. Datasets and Evaluation Protocol

We evaluate the method on six datasets covering binary and multi-class classification.

Binary datasets. *Two Moons* and *Circles* are 2-D synthetic datasets ($n=10,000$, $\sigma=0.1$ noise) generated via `scikit-learn` (Pedregosa et al., 2011) with a 70%/15%/15% train/val/test split. These datasets are useful for visualising the geometric structure of learned eigenfunctions. *HTRU2* (Lyon et al., 2016; Lyon, 2016) is a 9-feature pulsar candidate dataset (17,898 samples, 70%/15%/15% split, standardised).

Multi-class datasets. *MNIST* (LeCun & Cortes, 2010) (784-dim, 60k/10k official split, 10% val carve from train, $K=16$), *Fashion-MNIST* (Xiao et al., 2017) (same protocol, $K=16$), and *CIFAR-10* (Krizhevsky, 2009) represented by 2048-dim penultimate-layer features of an ImageNet-pre-trained ResNet-18 (50k/10k official split, 10% val carve, $K=16$).

All reported numbers are **validation accuracy** (mean $\pm$ std over 5 random seeds). No data augmentation ($\sigma=0$, $\rho=0$) and no Graph-Laplacian pre-training are used in the experiments. The network architecture is an MLP with hidden dimensions [64, 64, 64]; the metric MLPs use [64, 64].

3.2. Baselines

We compare against two families of baselines.

Classical baselines. Random Forest (RF, 500 trees) and Logistic Regression (LR) trained on raw features, both implemented in Scikit-Learn (Pedregosa et al., 2011).

NeuralEF (Deng et al., 2022) computes the leading eigenfunctions of a kernel integral operator via neural networks; we use the authors’ code with $K=16$ and report the best validation accuracy over 5 seeds (MNIST, Fashion-MNIST, CIFAR-10 only, trained on the same feature representation).

3.3. Main Results

Table 1 reports validation accuracy for all methods and datasets. The three EFDO variants are: **off** ($\mathbf{A}=\mathbf{I}$), **diag** ($\mathbf{A}=\mathbf{\Lambda}$), and **trotter** (full $\mathbf{A}=\mathbf{\Lambda}\mathbf{U}$, Trotter parametrisation). A $\dagger$ mark indicates results that are still being collected at submission time.

How to read the results. On the *linearly separable* HTRU2 dataset, both LR and EFDO-off match RF, confirming the method does not degrade when the raw features are already informative. On *Two Moons*, where LR achieves only 87.8% (the linear boundary cannot separate the two crescents), EFDO-off reaches 100.0%, demonstrating that spectral features capture the nonlinear structure of the data manifold. On *MNIST* and *CIFAR-10*, EFDO-off/diag substantially outperform NeuralEF (+12.0 pp and +9.4 pp respectively), indicating that joint optimisation of spectral features and the classification objective is more data-efficient than purely unsupervised eigenfunction learning. The *Circles* dataset exhibits high variance across seeds for all EFDO variants; this is a known issue with gradient-based optimisation on datasets with non-trivial topology and is a subject of ongoing investigation. The diagonal metric (DIAG) shows consistent improvement over OFF on Fashion-MNIST (+0.45 pp), suggesting that learned axis-aligned feature scaling is beneficial for heterogeneous pixel-based features.

3.4. Eigenfunction Visualisation

Figure 1 shows the first three learned eigenfunctions ϕ_1, ϕ_2, ϕ_3 on *Two Moons* and *Circles* (EFDO-off, seed 0). The eigenfunctions capture the geometric structure of the data: on *Two Moons*, ϕ_1 separates the two crescents; ϕ_2 and ϕ_3 encode finer intra-class structure. This confirms that the

Table 1. Validation accuracy (% , mean $\pm$ std, 5 seeds). Bold: best EFDO variant per row. †: results pending; *: NeuralEF reports test accuracy.

Dataset	RF	LR	NeuralEF*	EFDO-off	EFDO-diag	EFDO-trotter
Two Moons	99.77 $\pm$ 0.04	87.80 $\pm$ 0.00	—	100.00 $\pm$ 0.00	99.97 $\pm$ 0.04	99.99 $\pm$ 0.03
Circles	98.53 $\pm$ 0.11	50.40 $\pm$ 0.00	—	78.23 $\pm$ 14.90	79.16 $\pm$ 4.82	†
HTRU2	98.07 $\pm$ 0.07	98.14 $\pm$ 0.00	—	97.52 $\pm$ 0.08	97.48 $\pm$ 0.04	†
MNIST	97.12 $\pm$ 0.03	92.20	82.52 $\pm$ 0.29	94.53 $\pm$ 0.33	93.63 $\pm$ 0.34	†
Fashion-MNIST	—	—	77.12 $\pm$ 0.36	86.75 $\pm$ 0.31	87.20 $\pm$ 0.56	†
CIFAR-10	—	—	76.05 $\pm$ 0.55	85.50 $\pm$ 0.53	84.98 $\pm$ 0.33	†

minimisation of the Modified Dirichlet Energy produces representations that are geometrically meaningful rather than purely discriminative.

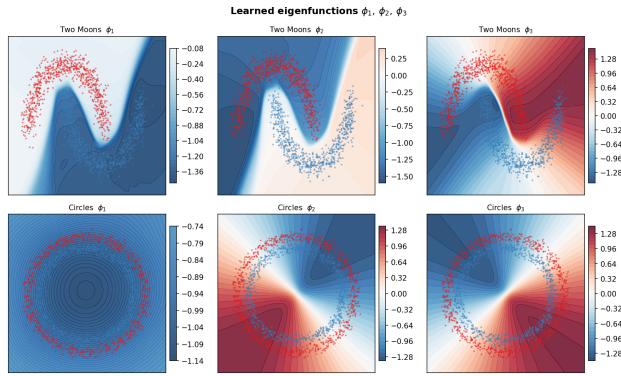


Figure 1. Learned eigenfunctions ϕ_1, ϕ_2, ϕ_3 for Two Moons (top) and Circles (bottom). Red/blue dots: class labels. Colour scale: eigenfunction value.

3.5. Training Dynamics

Figure 2 shows validation accuracy over training steps for MNIST and CIFAR-10 (EFDO-off and EFDO-diag, mean $\pm$ std over 5 seeds). Both variants converge rapidly in the first 15k steps and remain stable thereafter, with negligible variance across seeds. The diagonal metric consistently achieves slightly lower accuracy on MNIST while providing a small but consistent improvement on Fashion-MNIST.

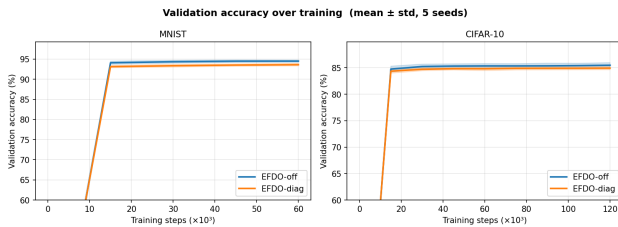


Figure 2. Validation accuracy over training (mean $\pm$ std, 5 seeds). Left: MNIST. Right: CIFAR-10.

3.6. Gram Matrix Analysis

Figure 3 shows the Gram matrix $C_K = \mathbb{E}[\Phi\Phi^T]$ computed on the MNIST validation set after training (EFDO-off, $K=16$, seed 0). The matrix is close to the identity, with Frobenius deviation $\|C - I\|_F < 0.5$ for the first eigenfunctions, confirming that the orthonormality loss (7) successfully enforces the constraint during sequential training.

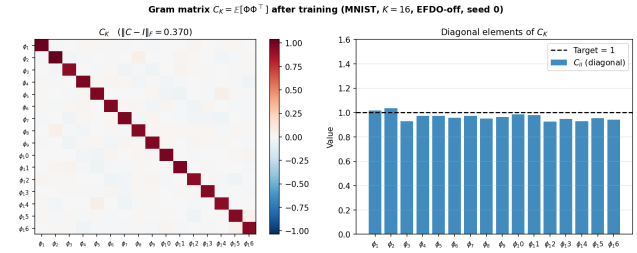


Figure 3. Gram matrix C_K (left) and diagonal elements (right) for MNIST EFDO-off, $K=16$, seed 0. Diagonal values close to 1 and small off-diagonal entries indicate near-orthonormal eigenfunctions.

We have performed all the simulations using PyTorch (Paszke et al., 2019) on Google Colab A100 GPUs (multiclass datasets: MNIST, Fashion-MNIST, CIFAR-10) and Apple M-series hardware (binary datasets). Training time per seed: ~ 2 min (Two Moons/HTRU2), ~ 8 min (MNIST, 60k steps), ~ 25 min (CIFAR-10, 120k steps) on A100. Approximation of a single eigen-function takes about two hours on CPU, and approximately 3–5 minutes per eigenfunction on GPU (MNIST, $d=784$). Therefore, computational time is one of the most essential disadvantages of the proposed approach on CPU. The solution to that problem could be appropriate initialization of the neural network, but it is the subject of future research.

Finally, it is important to notice that the warm-up stage is critical to avoiding local-minimum traps. The intuition behind the weighting introduced is based on the observation that the Dirichlet Energy is high for strongly oscillating functions. At the same moment, highly oscillating functions are likely to have non-zero projection on the linear subspace of functions orthogonal to smooth eigenfunctions that have been approximated already. Therefore, by promoting oscilla-

tions and orthogonality at the beginning of training, we can increase the capability of ANN for detection of orthogonal directions in a function space.

4. Discussion and Concluding Remarks.

In the current work we present the novel sequential approach for approximation of Laplacian-like differential operators. The central idea of the method is a two-stage procedure that allows the ANN to escape from the local minimum and find the direction orthogonal to the space spanned by previously constructed eigenfunctions. We achieve that by introducing the instability to the optimization process that deforms the ANN approximation of eigen function to such a degree that novel direction can be discovered.

This instability stage or warm-up stage is followed by optimization of the loss function with positive weights with a certain hierarchy. That weights are constructed in such a way that ANN tries to satisfy orthogonalization and normalization constraints first before minimizing the objective function, which is the Dirichlet Energy. This idea is quite generic and can be directly applied to training of other ANNs with constraints.

In addition to the novel ANN training algorithm, we study the problem of overfitting and ANN performance on outliers. To address that issue, two data augmentation techniques have been proposed. That regularization techniques can be considered as a perturbation of the probability distribution from which the data has been sampled. The degree of that disturbance is adjusted in such a way that the disturbed probability distribution goes to the original probability distribution as the number of samples goes to infinity.

Finally, we demonstrate on the datasets with different numbers of samples that the proposed approach for calculation of eigenfunctions provides meaningful results at least from a machine learning point of view. The accuracy of models trained on the eigenfunction values is similar to the accuracy of the models trained on the original data. Moreover, in some cases training with eigenfunction values requires less samples to reach the similar accuracy in comparison with training on the original data. Therefore, the presented approach can be adopted as a feature-engineering tool for machine learning tasks. One of the main issues with the proposed approach is high computational cost. However, it might be resolved with appropriate initialization strategy, which is a subject of future research.

References

Ahmadi, A. and Patras, I. Unsupervised convolutional neural networks for motion estimation. In *2016 IEEE international conference on image processing (ICIP)*, pp.

1629–1633. IEEE, 2016.

Ajmal, H., Rehman, S., Farooq, U., Ain, Q. U., Riaz, F., and Hassan, A. Convolutional neural network based image segmentation: a review. *Pattern Recognition and Tracking XXIX*, 10649:191–203, 2018.

Amato, F., López, A., Peña-Méndez, E. M., Vaňhara, P., Hampl, A., and Havel, J. Artificial neural networks in medical diagnosis, 2013.

Belkin, M. and Niyogi, P. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008. ISSN 0022-0000. doi: <https://doi.org/10.1016/j.jcss.2007.08.006>. URL <https://www.sciencedirect.com/science/article/pii/S0022000007001274>. Learning Theory 2005.

Ben-Shaul, I., Bar, L., Fishelov, D., and Sochen, N. Deep Learning Solution of the Eigenvalue Problem for Differential Operators. *Neural Computation*, 35(6):1100–1134, 05 2023. ISSN 0899-7667. doi: 10.1162/neco.a_01583. URL https://doi.org/10.1162/neco.a_01583.

Calder, J., Cook, B., Thorpe, M., and Slepcev, D. Poisson learning: Graph based semi-supervised learning at very low label rates. In *Proceedings of the International Conference on Machine Learning*, pp. 8588—8598, 2020.

Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 2023.

Choo, K., Mezzacapo, A., and Carleo, G. Fermionic neural-network states for ab-initio electronic structure. *Nature Communications*, 11, 05 2020. doi: 10.1038/s41467-020-15724-9.

Deng, Z., Shi, J., and Zhu, J. NeuralEF: Deconstructing kernels by deep neural networks. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 4976–4992. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/deng22b.html>.

Evans, L. C. *Partial differential equations*, volume 19. American Mathematical Society, 2022.

Feld, T., Aujol, J.-F., Gilboa, G., and Papadakis, N. Rayleigh quotient minimization for absolutely one-homogeneous functionals. *Inverse Problems*, 35(6):064003, may 2019. doi: 10.1088/1361-6420/ab0cb2. URL <https://dx.doi.org/10.1088/1361-6420/ab0cb2>.

- Ford, W. Chapter 5 - eigenvalues and eigenvectors. In Ford, W. (ed.), *Numerical Linear Algebra with Applications*, pp. 79–101. Academic Press, Boston, 2015. ISBN 978-0-12-394435-1. doi: <https://doi.org/10.1016/B978-0-12-394435-1.00005-3>. URL <https://www.sciencedirect.com/science/article/pii/B9780123944351000053>.
- Gomez-Chova, L., Camps-Valls, G., Munoz-Mari, J., and Calpe, J. Semisupervised image classification with laplacian support vector machines. *IEEE Geoscience and Remote Sensing Letters*, 5(3):336–340, 2008. doi: 10.1109/LGRS.2008.916070.
- Halko, N., Martinsson, P. G., and Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806. URL <https://doi.org/10.1137/090771806>.
- Jin, H., Mattheakis, M., and Protopapas, P. Unsupervised neural networks for quantum eigenvalue problems. *ArXiv*, abs/2010.05075, 2020. URL <https://api.semanticscholar.org/CorpusID:222290530>.
- Kostopoulos, G., Karlos, S., Kotsiantis, S., and Ragos, O. Semi-supervised regression: A recent review. *Journal of Intelligent & Fuzzy Systems*, 35(2):1483–1500, 2018.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Kunegis, J., Schmidt, S., Lommatzsch, A., Lerner, J., Luca, E. W. D., and Albayrak, S. *Spectral Analysis of Signed Graphs for Clustering, Prediction and Visualization*, pp. 559–570. doi: 10.1137/1.9781611972801.49. URL <https://epubs.siam.org/doi/abs/10.1137/1.9781611972801.49>.
- Lagaris, I., Likas, A., and Fotiadis, D. Artificial neural network methods in quantum mechanics. *Computer Physics Communications*, 104(1):1–14, 1997. ISSN 0010-4655. doi: [https://doi.org/10.1016/S0010-4655\(97\)00054-4](https://doi.org/10.1016/S0010-4655(97)00054-4). URL <https://www.sciencedirect.com/science/article/pii/S0010465597000544>.
- Le, T. H. Applying artificial neural networks for face recognition. *Advances in Artificial Neural Systems*, 2011, 2011.
- LeCun, Y. and Cortes, C. MNIST handwritten digit database. 2010. URL <http://yann.lecun.com/exdb/mnist/>.
- Lyon, R. HTRU2. 3 2016. doi: 10.6084/m9.figshare.3080389.v1. URL <https://figshare.com/articles/dataset/HTRU2/3080389>.
- Lyon, R. J., Stappers, B. W., Cooper, S., Brooke, J. M., and Knowles, J. D. Fifty years of pulsar candidate selection: from simple filters to a new principled real-time classification approach. *Monthly Notices of the Royal Astronomical Society*, 459(1):1104–1123, 04 2016. ISSN 0035-8711. doi: 10.1093/mnras/stw656. URL <https://doi.org/10.1093/mnras/stw656>.
- M. A. ERICKSON, R. S. S. and LAUB, A. J. Power methods for calculating eigenvalues and eigenvectors of spectral operators on hilbert spaces. *International Journal of Control*, 62(5):1117–1128, 1995. doi: 10.1080/00207179508921586. URL <https://doi.org/10.1080/00207179508921586>.
- Ng, A., Jordan, M., and Weiss, Y. On spectral clustering: Analysis and an algorithm. In Dietterich, T., Becker, S., and Ghahramani, Z. (eds.), *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001. URL https://proceedings.neurips.cc/paper_files/paper/2001/file/801272ee79cfde7fa5960571fee36b9b-Paper.pdf.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32, pp. 8024–8035. Curran Associates, Inc., 2019. URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance.pdf>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. ISSN 0033-295X. doi: 10.1037/h0042519. URL <http://dx.doi.org/10.1037/h0042519>.
- Samoshyn, A. Dataset of songs in spotify. 2010. URL <https://www.kaggle.com/datasets/mrmorj/dataset-of-songs-in-spotify/data>.
- Sejnowski, T. J. *6 The Cocktail Party Problem*, pp. 81–90. 2018.

Sheikhpour, R., Sarram, M. A., and Sheikhpour, E. Semi-supervised sparse feature selection via graph laplacian based scatter matrix for regression problems. *Information Sciences*, 468:14–28, 2018.

Simon, J. B., Dickens, M., and Deweese, M. Neural tangent kernel eigenvalues accurately predict generalization, 2022. URL <https://openreview.net/forum?id=lycl1GD7fVP>.

Xiao, H., Rasul, K., and Vollgraf, R. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL <https://arxiv.org/abs/1708.07747>.

A. You *can* have an appendix here.

You can have as much text here as you want. The main body must be at most 8 pages long. For the final version, one more page can be added. If you want, you can use an appendix like this one.

The `\onecolumn` command above can be kept in place if you prefer a one-column appendix, or can be removed if you prefer a two-column appendix. Apart from this possible change, the style (font size, spacing, margins, page numbering, etc.) should be kept the same as the main body.